

ICT3113 Assignment 1 (26 Sept 2026)

Choose Candidate Models, Set Requirements, and Record your Predictions

1. Candidate Models

Model	Ollama Tag	Params	Size Class	Digest
Gemma 3	gemma3:1b	1B	Small	sha256:7cd4618c1faf8b7233c6c906dac1694b6a47684b37b8895d470ac688520b9c01
Gemma 3	gemma3:4b	4B	Medium	sha256:aeda25e63ebd698fab8638ffb778e68bed908b960d39d0becc650fa981609d25
Qwen 2.5	qwen2.5:3b	3B	Medium	sha256:5ee4f07cdb9beadbbb293e85803c569b01bd37ed059d2715faa7bb405f31caa6
Qwen 2.5	qwen2.5:7b	7B	Large	sha256:2bada8a7450677000f678be90653b85d364de7db25eb5ea54136ada5f3933730
Granite 3.3	granite3.3:8b	8B	Large	sha256:77bcee066a76dcd10d0d123c87e32c8ec2c74e31b6ffd87ebee49c9ac215dca

Our chosen candidate models Spans three size classes (1B/3-4B/7-8B), all CPU-only, per the client's no-GPU constraint. Two same-family pairs: gemma3:1b→4b and qwen2.5:3b→7b isolate the effect of scale from vendor/training differences: since architecture and training approach are held constant within each pair, any accuracy or latency change can only be attributed to parameter count. granite3.3:8b is included on a separate, distinct rationale: IBM markets the Granite 3.x family specifically for classification and extraction tasks rather than general-purpose chat, making it a deliberate test of whether that training emphasis outperforms a similarly-sized general-purpose model (qwen2.5:7b).

2. Requirements

2.1. Response Time

$p_{95} \leq 5s$, $p_{99} \leq 10s$ at peak load

2.2. Throughput

≥ 0.016 req/s (~58 tickets/hr) sustained without breaching latency

2.3. Accuracy

$\geq 85\%$ overall, $\geq 70\%$ floor per category

3. Prediction Records

3.1. Bottleneck

We predict the **Ollama model backend to be the throughput bottleneck.**

As per the Service Demand Law ($U=X*D$), the model backend is the only single-instance CPU bound component in the pipeline with no concurrency.

SQLite writes and Express routing complete in low single digit milliseconds, while model inference is measured in seconds. The device with the largest service demand and no replication sets the throughput ceiling ($X \leq 1/D$), and that is Ollama in every candidate configuration selected.

Specifically, we predict that for the 7-8B candidates (qwen2.5:7b & granite3.3:8b), the system will fail to sustain the 0.016 req/s throughput requirement once concurrent requests exceed 2-3 simultaneous tickets, because Ollama's default CPU configuration process requests through a single model instance rather than true parallel inference. We expect the stress test (in step 5) to show queuing and rising p95/p99 latency as concurrency increases for these two models, while gemma3:1b, gemma3:4b, and qwen2.5:3b sustain the requirement comfortably at the same concurrency levels.

3.2. Expected accuracy and latency

Latency predictions are informed by a real measurement that we took when building the baseline service (step 2). The model used (llama3.2:1b) is not a candidate, and revealed a cold-start/warm-start split that has direct consequences for this workload

```
service.log
{"timestamp":"2026-09-26T08:01:52.369Z","type":"startup","message":"ticket-triage-service listening on port 3000"}
{"timestamp":"2026-09-26T08:08:37.795Z","type":"error","path":"/tickets","message":"Expected property name on '}' in JSON at position 1","stack":"SyntaxError: Expect
{"timestamp":"2026-09-26T08:09:08.672Z","type":"classification","ticket_id":1,"category":"Bank account or service","model":"llama3.2:1b","classification_ms":2808,"fa
{"timestamp":"2026-09-26T08:09:08.673Z","type":"request","method":"POST","path":"/tickets","status":201,"duration_ms":2812}
{"timestamp":"2026-09-26T08:09:54.111Z","type":"request","method":"GET","path":"/stats","status":200,"duration_ms":2}
{"timestamp":"2026-09-26T13:22:10.637Z","type":"classification","ticket_id":2,"category":"Bank account or service","model":"llama3.2:1b","classification_ms":3857,"f
{"timestamp":"2026-09-26T13:22:10.643Z","type":"request","method":"POST","path":"/tickets","status":201,"duration_ms":3881}
{"timestamp":"2026-09-26T13:23:02.699Z","type":"classification","ticket_id":3,"category":"Bank account or service","model":"llama3.2:1b","classification_ms":208,"fa
{"timestamp":"2026-09-26T13:23:02.700Z","type":"request","method":"POST","path":"/tickets","status":201,"duration_ms":211}
{"timestamp":"2026-09-26T13:23:06.341Z","type":"classification","ticket_id":4,"category":"Bank account or service","model":"llama3.2:1b","classification_ms":200,"fa
{"timestamp":"2026-09-26T13:23:06.341Z","type":"request","method":"POST","path":"/tickets","status":201,"duration_ms":204}
```

Figure 1. Service log snippet showing cold-start vs warm-state classification latency (for llama3.2:1b)

Ticket	Gap since last request	classification_ms	State
1	NIL	2,808	Cold
2	~5h13m	3,857	Cold
3	52s	208	Warm
4	4s	200	Warm

Ollama's default "OLLAMA_KEEP_ALIVE" unloads a model from memory after 5 minutes of inactivity. The next request then pays a full reload from disk penalty. This is relevant to our workload model. Our baseline volume (~110 tickets/day) implies an average time of ~13 minutes, which is longer than the default 5 minute keep-alive. Under non-peak conditions, most production requests would hit a cold start, not the warm-state latency.

We have set "OLLAMA_KEEP_ALIVE= -1" in docker-compose.yml, to keep the model in memory indefinitely once loaded. This means the cold start penalty occurs only once per container start rather than repeatedly during normal operation. Warm latency is therefore the correct assumption for our response time requirement, cold start is tracked separately as a one-time startup cost, not a recurring per ticket risk

Model	Predicted Accuracy	Warm Latency	Cold Latency
gemma3:1b	60-68%	150-350ms	2500-4000ms
gemma3:4b	72-80%	500-1000ms	6000-10000ms
qwen2.5:3b	68-76%	400-800ms	4000-7000ms
qwen2.5:7b	82-88%	1000-2000ms	10000-16000ms
granite3.3:8b	84-90%	1200-2500ms	12000-18000ms

Warm and cold figures are scaled from “llama3.2:1b” baseline model’s latency. Neither scaling is validated beyond the one data point. Smaller models are expected to make more classification errors, especially on tickets that don’t explicitly name their financial product (pattern identified when labelling golden test set). We expect this ambiguity to hurt small models disproportionately.

Within each same family pair, we predict the larger model outperforms the smaller one on accuracy but loses on latency, which is the core tradeoff this candidate set is designed to make visible.

We predict granite3.3:8b slightly outperforms the similarly sized qwen2.5:7b on accuracy specifically, on the strength of its stated classification training emphasis. This is a deliberate test of whether that training emphasis actually matters against a general purpose model of similar scale.

3.3. Which categories is expected to be hardest to classify

We predict “**Bank account or service**” and “**Money transfer or service**” will be the hardest pair for every candidate model to distinguish.

This prediction is directly evidenced by our golden set construction. When the two labelers worked from a written protocol with explicit edge-case rules, their agreement was the lowest on exactly this boundary (specific agreement of 0.69-0.76 for Bank account/Money transfer, compared to 0.90+ for most other categories).

It took our team 10 rounds of protocol revision to converge on a stable rule for this exact boundary. We predict LLMs, given only a category list and no equivalent of our detailed edge-case protocol, will struggle at least as much as our team’s labelers did on their first independent pass, but likely more since the models have no access to our specific tie-break rules.

TLDR:

Bottleneck: Ollama is the throughput bottleneck

Accuracy and Latency (warm state):

Model	Predicted Accuracy	Warm Latency
gemma3:1b	60-68%	150-350ms
gemma3:4b	72-80%	500-1000ms
qwen2.5:3b	68-76%	400-800ms
qwen2.5:7b	82-88%	1000-2000ms
granite3.3:8b	84-90%	1200-2500ms

Hardest Categories: “Bank account or service” vs “Money transfer or service”